

---

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

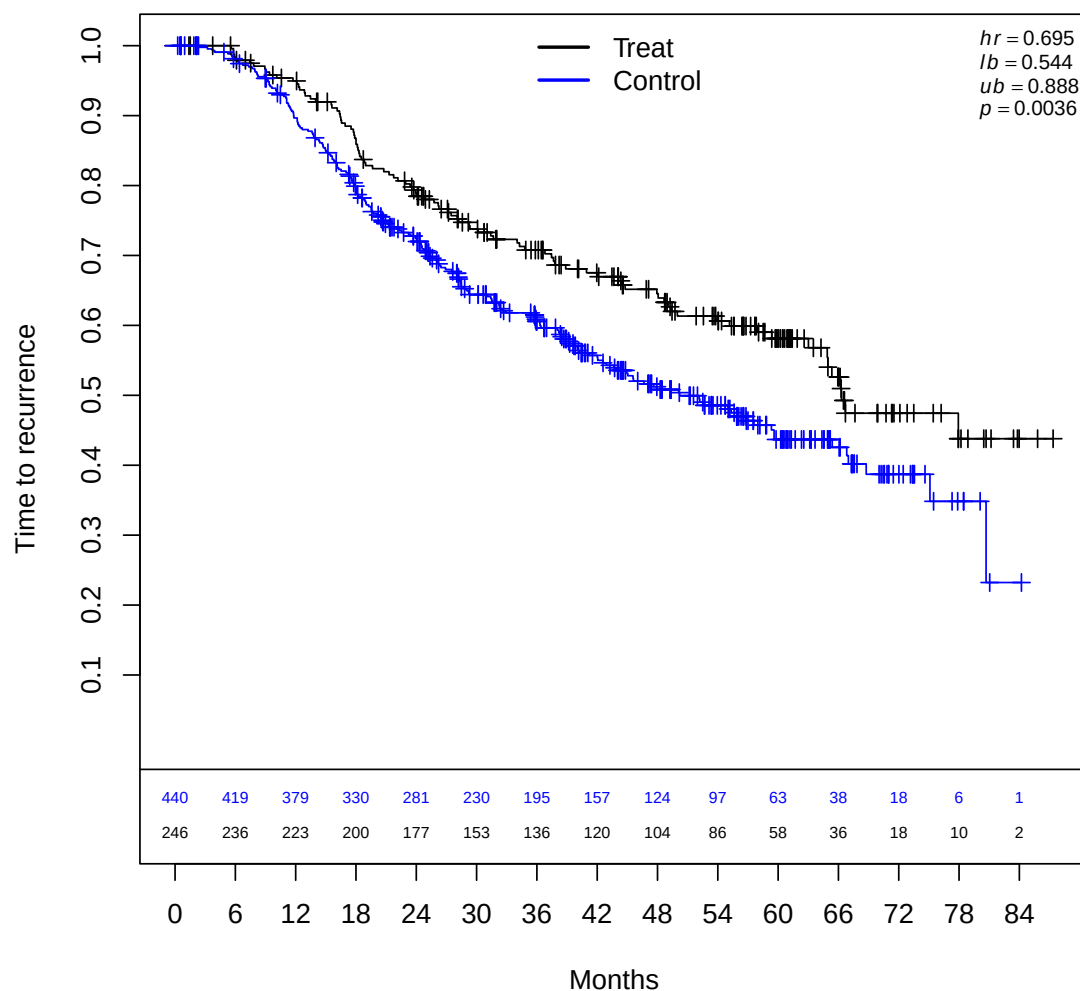
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2.5
# out.loc = NULL turns off file creation
N <- 700
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.499996 0.6466405
## Causal HR (empirical ITT)= 0.7125876
```

```
file_out <- c("output/sim-FS4_Lasso_ex9.Rdata")

n_add_noise <- 3

# sim=69 sim=71 sim <- 101

sim <- 75

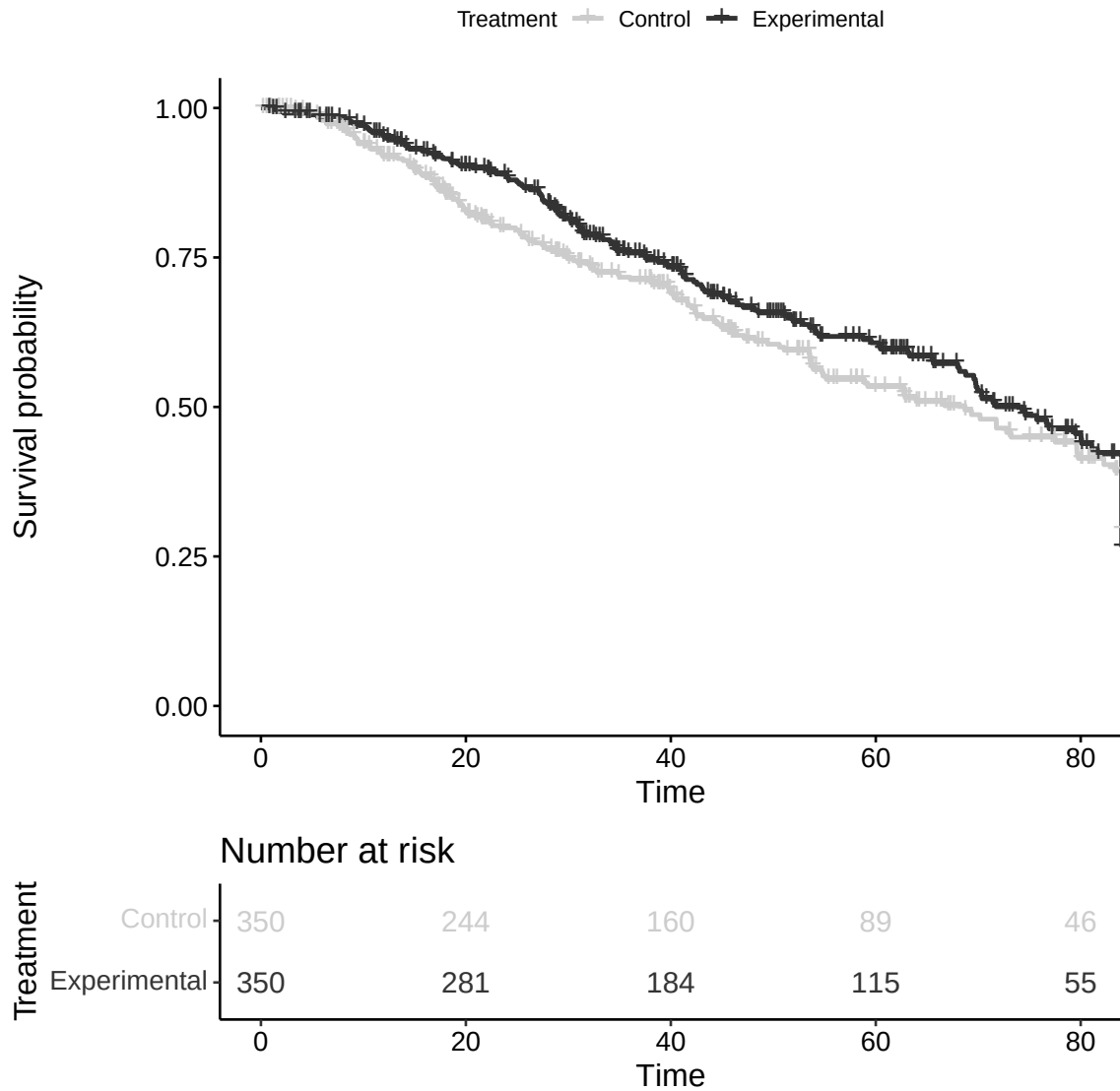
dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
```

```
legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
risk.table = TRUE, risk.table.col = "strata")
```



```
if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3", "noise4", "noise5")
}
```

```

}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)
  # Add 3 noise
  x$noise1 <- rnorm(N, sd = 1)
  x$noise2 <- rnorm(N, sd = 1)
  x$noise3 <- rnorm(N, sd = 1)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade", "noise1",
    "noise2", "noise3")
}

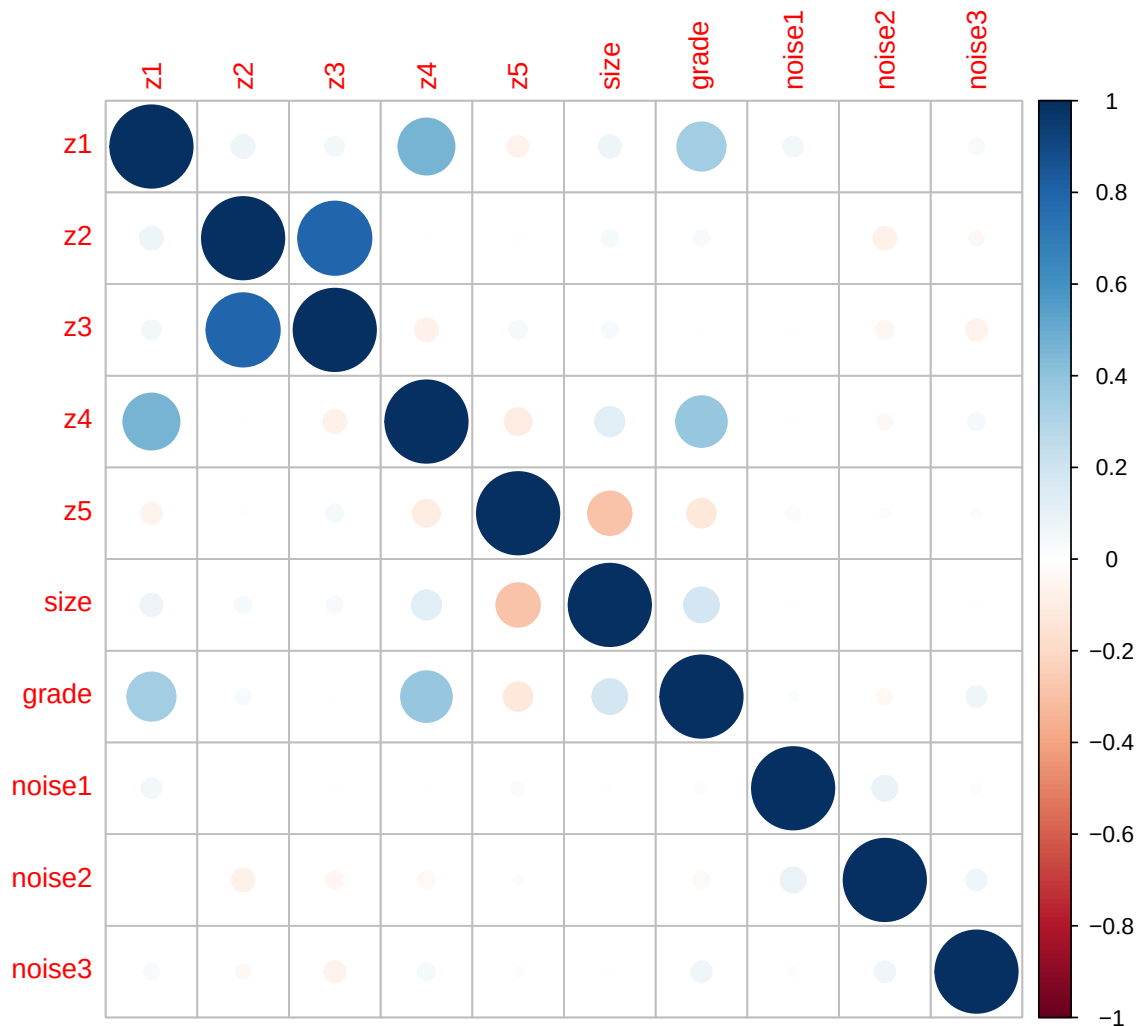
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade+noise1+
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -0.197284  0.820957  0.104956 -1.880 0.060152
## v11      0.263835  1.301913  0.126180  2.091 0.036533
## v21     -0.601344  0.548075  0.180358 -3.334 0.000856
## v31      0.524441  1.689514  0.183815  2.853 0.004330
## v41      0.885843  2.425029  0.126098  7.025 2.14e-12
## v51     -0.848301  0.428142  0.110317 -7.690 1.48e-14
## size    -0.002583  0.997421  0.003798 -0.680 0.496486
## grade    0.119183  1.126576  0.102914  1.158 0.246832
## noise1   0.013609  1.013702  0.050540  0.269 0.787718
## noise2  -0.043224  0.957697  0.051025 -0.847 0.396935
## noise3  -0.034119  0.966457  0.052343 -0.652 0.514510
##
## Likelihood ratio test=181.7 on 11 df, p=< 2.2e-16
## n= 700, number of events= 383

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- FALSE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade
## 10 x 2 sparse Matrix of class "dgCMatrix"
##           s0      s0
## z1      0.253050853 0.301188117
## z2     -0.278132648 0.331041163
## z3      0.206750105 0.246079688
## z4      0.840175419 1.000000000
## z5     -0.756346805 0.900224867
## size      .      .
## grade    0.051799456 0.061653144
## noise1     .      .
## noise2 -0.009730472 0.011581477
## noise3 -0.007912136 0.009417243
## Cox-LASSO selected: z1 z2 z3 z4 z5 grade
## Cox-LASSO not selected: size noise1 noise2 noise3
## Median cuts (after Lasso if use_lasso)
## Categorical (after Lasso if use_lasso) z1 z2 z3 z4 z5 grade
## # of candidate subgroup factors= 6
## [1] "z1"  "z2"  "z3"  "z4"  "z5"  "grade"
## Confounders per grf screening q1 q6 q5 q2 q4 q3
##   FSconfounders.name      vi.cs vi_ratio
## 1                q1 0.40078477 2.4047086
## 6                q6 0.21066726 1.2640035
## 5                q5 0.11428710 0.6857226
## 2                q2 0.10453458 0.6272075
## 4                q4 0.09475119 0.5685071
## 3                q3 0.07497510 0.4498506
## Number of unique levels (L) and possible subgroups= 13 8191
## # of subgroups based on # variables > k.max and excluded (per million) 0.0081
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 19 15
## # of subgroups with sample size less than criteria 18
## # of subgroups meeting all criteria = 72
## # of subgroups fitted (Cox model estimable) = 72
## *Subgroup Searching Minutes=* 0.003833333
## Number of subgroups meeting HR threshold 13
## # of candidate subgroups (meeting HR criteria) = 13
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##   K  n   E d1   m1   m0  HR L(HR) U(HR) q1.0 q1.1 q6.1 q6.2 q6.3 q5.0
## 1: 2 100 75 38 22.43 39.90 1.99 1.25 3.16 0 1 0 0 0 0
## 2: 1 87 32 20 73.37 NA 1.90 0.93 3.91 0 0 1 0 0 0
## 3: 2 79 27 17 74.61 NA 1.85 0.85 4.06 0 0 1 0 0 0
## 4: 2 85 32 20 73.37 NA 1.83 0.89 3.77 1 0 1 0 0 0
## 5: 2 91 56 30 27.45 53.52 1.81 1.07 3.07 0 1 0 0 0 0
## 6: 2 155 114 60 24.16 38.37 1.73 1.19 2.51 0 1 0 0 0 0
## 7: 2 88 53 27 31.55 51.38 1.68 0.98 2.89 0 1 0 0 0 0
## 8: 2 79 63 36 23.12 25.30 1.68 1.01 2.80 0 1 0 0 1 0

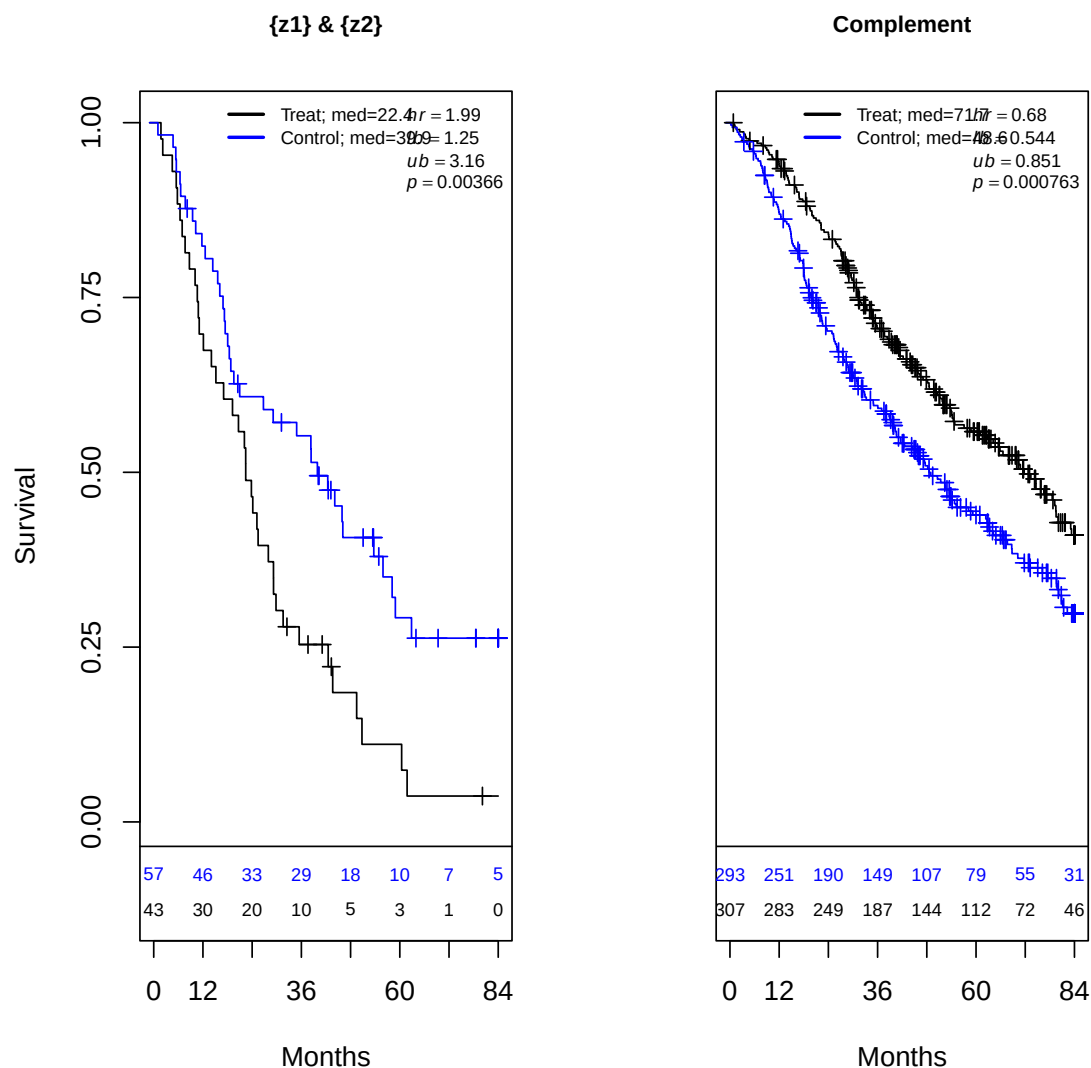
```

---

```

## 9: 1 172 124 65 24.16 38.97 1.64 1.15 2.34 0 1 0 0 0 0
## 10: 2 81 68 35 22.43 34.85 1.59 0.97 2.58 0 1 0 0 0 0
##      q5.1 q2.0 q2.1 q4.0 q4.1 q3.0 q3.1
## 1: 0 0 1 0 0 0 0
## 2: 0 0 0 0 0 0 0
## 3: 0 0 0 1 0 0 0
## 4: 0 0 0 0 0 0 0
## 5: 0 0 0 0 0 1 0
## 6: 0 0 0 0 1 0 0
## 7: 1 0 0 0 0 0 0
## 8: 0 0 0 0 0 0 0
## 9: 0 0 0 0 0 0 0
## 10: 0 0 0 0 0 0 1
## Consistency 1
## # of splits= 400
## Model, % Consistency Met= {z1} {z2} 1
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K M.1 M.2
## 1: 1 100 9 1 2 {z1} {z2}

```



```
## [1] "{z1}" "{z2}"
## % consistency criteria met= 1
## SG focus= hr
## Subgroup Consistency Minutes= 0.02593333
## Subgroup found
## Minutes overall= 0.04185

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 82.25797 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -8.739154   528.000000   2.333029     8.739154   528.000000
## 2         3    10.390022   172.000000   4.173258    -10.390022   172.000000
## 3         6   -11.381147   443.000000   2.528517    11.381147   443.000000
## 4         7    10.034508   170.000000   4.213843    -10.034508   170.000000
##   treated.se      diff Nsg depth
## 1    2.333029  -17.47831 528     1
```

```
## 2  4.173258 20.78004 172 1
## 3  2.528517 -22.76229 443 2
## 4  4.213843 20.06902 170 2
## leaf.node control.mean control.size control.se treated.mean treated.size
## 2      3      10.390022      172.000000      4.173258      -10.390022      172.000000
## treated.se      diff Nsg depth
## 2  4.173258 20.78004 172 1
## [1] "z1 <= 0"
## [1] "z1 <= 0"
```

---

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 1.3969

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 1.3969

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
## Propn bootstrap subgroups found = 0.958
```

```

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
    "\n")

## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 1.986355 0.4691248 1.250333 3.155646 1.652596 0.4777877 0.9377194 2.912464

print(Hc_estimates)

##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower
## 1: 0.680257 0.07786825 0.5435468 0.8513519 0.6927555 0.1064293 0.5126351
##           H1_upper
## 1: 0.9361634

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")

## Overall minutes for analysis 1.5179

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# dfH <- subset(fs.est$df.est, flag.harm==1) dfH_est <- subset(fs.est$df.est,
# treat.recommend==0) dfSens <- subset(fs.est$df.est, flag.harm==1 &
# treat.recommend==0) sensH <- nrow(dfSens)/nrow(dfH)

## Confounders evaluated in Forestsearch:
## [1] "z1" "z2" "z3" "z4" "z5" "grade"

```

**Table 1** Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups  $H$  and  $H^c$ . Cox model estimates are based on subgroups:  $H$  true (knowing the actual subgroup, a-priori); the estimated subgroup  $\hat{H}$ ; and the bootstrap (1,000 resamples) bias-correction to  $\hat{H}$  estimates, denoted  $\hat{H}_{bc}$ . Estimates for the complement  $H^c$  are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups  $H$  and  $H^c$ , respectively, are listed.

|                                        | HR Estimate | Lower | Upper | # Subjects | # Correct |
|----------------------------------------|-------------|-------|-------|------------|-----------|
| <b>ITT</b>                             |             |       |       |            |           |
| ITT                                    | 0.778       | 0.636 | 0.951 | 700        | .         |
| <b>H subgroup estimates</b>            |             |       |       |            |           |
| $H_{oracle}$                           | 1.585       | 0.973 | 2.583 | 81         | .         |
| $\hat{H}$                              | 1.986       | 1.25  | 3.155 | 100        | 81        |
| $\hat{H}_{bc}$                         | 1.653       | 0.938 | 2.914 | 100        | 81        |
| <b>H-complement subgroup estimates</b> |             |       |       |            |           |
| $H_{oracle}^c$                         | 0.717       | 0.575 | 0.895 | 619        | .         |
| $\hat{H}^c$                            | 0.68        | 0.543 | 0.851 | 600        | 600       |
| $\hat{H}_{bc}^c$                       | 0.693       | 0.513 | 0.935 | 600        | 600       |